$R(8,10) \geq 346$: a colouring of K_{345} with no red K_8 and no blue K_{10}

David Lai
davidlai911@gmail.com

3 November 2026

Abstract

We describe an edge colouring of the complete graph K_{345} containing no red K_8 and no blue K_{10}, which proves $R(8,10) \geq 346$. The previous bound, $R(8,10) \geq 345$, is due to Cremer [1], improving the value 343 recorded in revision 18 of Radziszowski's dynamic survey [2]. Our colouring is obtained by the same device, applied once more: add one vertex to Cremer's colouring of K_{344} and repair the monochromatic cliques that appear. The added vertex creates 29 814 blue K_{10}'s, and the repair reduces that count to zero. We report the colouring, four independent verifications of it, and the search trajectory, including the one element of the search that appeared to be decisive: restarting from the best colouring found so far with fresh random seeds, round after round.

1 Introduction

Write $R(r,s)$ for the least n such that every red/blue colouring of the edges of K_n contains a red K_r or a blue K_s. An (r,s)-*colouring* of K_n is one containing neither, and shows $R(r,s) \geq n+1$. Table 2.1 of Radziszowski's dynamic survey [2], in its current revision, lists

$$343 \;\leq\; R(8,10) \;\leq\; 4402.$$

The lower bound 343 is due to Kuznetsov [3], whose witness is a $(8,10)$-colouring of K_{342} that is circulant: $\{i,j\}$ is red exactly when $\min(|i-j|, 342-|i-j|)$ lies in a fixed set of 76 distances. Machine search has supplied a number of nearby entries in the same table; Nagda, Raghavan and Thakurta [5], for instance, improved nine bounds of the form $R(3,s)$ and $R(4,s)$. Cremer [1] observed that adding a single vertex to that colouring, joined by a blue edge, produces 10 146 blue K_{10}'s, and that local search can remove all of them; the resulting colouring of K_{343} proves $R(8,10) \geq 344$. Adding one further vertex and repeating the process gives a colouring of K_{344} and the bound $R(8,10) \geq 345$ [1].

This note records the next step. Starting from Cremer's colouring of K_{344}, adding one vertex creates 29 814 blue K_{10}'s — the exact minimum over all $2 \cdot 344$ choices of which vertex to copy and which colour to give the joining edge — and exact-conflict local search reduces that number to zero. The result is a $(8,10)$-colouring of K_{345}:

Theorem. $R(8,10) \geq 346$.

Section 2 states the colouring and its basic invariants; Section 3 records four independent verifications; Section 4 describes how the colouring was found and what in the search mattered; Section 5 describes the AI system that ran the search and what it contributed; Section 6 says

what the computation does not establish; and Section 7 lists the artefacts needed to reproduce the result; and Appendix A records an exact bounded-distance screen of the same one-vertex step on six smaller cells, which shows how far a repair must travel before it can succeed at all.

Provenance. The search code was written and run by an AI agent under the author's direction, following the same arrangement described in Section 5 of [1]. The screening stage was a multi-agent run; the final descent was not. Section 5 describes both, and what each contributed. Every number in this note was regenerated and checked independently of the search process before being recorded here; Section 3 specifies the checkers, and Section 7 lists the artefacts needed to reproduce or refute the result.

2 The colouring

The colouring is stored as a 345×345 matrix M with entries in $\{0, 1\}$, symmetric, with zero diagonal, where $M_{ij} = 1$ means that $\{i, j\}$ is red. Its invariants are as follows.

vertices	345
red edges	26 045
blue edges	33 295
ω_{red} (largest red clique)	7
ω_{blue} (largest blue clique)	9
red degrees	146 to 160, mean 150.99
distinct red degrees	15

Since $\omega_{\text{red}} = 7 < 8$ and $\omega_{\text{blue}} = 9 < 10$, the colouring contains no red K_8 and no blue K_{10}, so $R(8, 10) \geq 346$.

As in [1] we note that the red graph is not circulant: its red degrees take 15 distinct values, so no relabelling makes it invariant under a cyclic shift. Nor is it a near-copy of any colourings the search passed through: it differs from the 6-conflict state it was built from in 1 032 edges and from a 1-conflict state found alongside it in 1 226 (Section 4).

3 Verification

The colouring was verified by four programs, written independently of one another and of the search code.

checker	method	result
`count`	exhaustive enumeration of red K_8, blue K_{10}	0, 0
`verify`	maximum-clique branch and bound	$\omega_{\text{red}} = 7$, $\omega_{\text{blue}} = 9$
`verify2`	target-size DFS, independent code	no red K_8, no blue K_{10}
`enum`	independent clique enumerator	0 conflicts

The two count-based checkers agree with the two structural ones: a colouring with $\omega_{\text{red}} = 7$ and $\omega_{\text{blue}} = 9$ contains no forbidden clique, and the counts confirm the same directly. To indicate the cost of an independent re-check, `verify2` searched 10 471 027 nodes for a red K_8 and 195 066 549 for a blue K_{10} on a single core; the blue search is two orders of magnitude larger, as expected when the forbidden blue clique is two vertices bigger.

The files were also decoded from their graph6 representation [6] and compared edge by edge with the matrix form, so that the two deposited encodings are known to carry the same graph.

4 How the colouring was found

The setting. A *conflict* is a monochromatic clique of a forbidden size: a red K_8 or a blue K_{10}. The engine used here was written during the screening stage described in §5. The search keeps the complete set of conflicts, together with the number of conflicts lying on each edge, so that the effect of flipping any edge is computed exactly rather than sampled. One iteration chooses a conflict at random with probability proportional to $w\,|K|$ over the two colours, samples up to 32 of its edges, and flips the edge of least weighted exact change δ, accepting it if $\delta \le 0$ or with Metropolis probability $\exp(-\delta/T)$, where $T = 0.25 + \min(2.5, t/100)$ and t counts iterations since the last improvement; the flipped edge is made tabu for 5 to 19 iterations. The weights are $w_{\text{red}} = \binom{10}{2}/\binom{8}{2} = 45/28$ and $w_{\text{blue}} = 1$. When no improvement has occurred for 1 000 iterations, the search restores the best colouring found so far, performs eight conflict-directed random flips, clears the tabu list and re-randomises the red weight; in addition, a *patch* move occasionally randomises the colours of pairs inside a set of up to eight vertices chosen around a vertex with probability proportional to its conflict count, and then repairs freely.

The first step, exactly. Copying a vertex v and joining the copy v' to v by an edge of colour c creates a conflict $\{v, v'\} \cup C$ for each c-clique C of size 6 (for red) or 8 (for blue) inside $N_c(v)$, and nothing else. One clique pass per vertex and colour therefore gives all $2n$ twin costs exactly. For Cremer's K_{344} colouring the minimum over all 688 choices is 29 814, attained by the blue twin of vertex 335; the cheapest red twin costs 41 479. The spread between twins is large, so this choice is worth making exactly.

The descent. From 29 814 conflicts the search descended as follows. Each row is a round of one or more single-threaded segments; within a round the best colouring of the previous round is the input, and the random seeds are new.

conflicts	round	lanes	wall clock
29 814 → 97	pilot, 13 segments, 11 seeds	1	≈ 102 CPU-minutes
97 → 96	single 90-second segment	1	90 s
96 → 41	round 1	4	≈ 2 h
41 → 23	round 2	4	≈ 2 h
23 → 6	round 3	4	≈ 2 h
6 → 0	round 4	4	≈ 2 h

The colouring that achieves zero conflicts comes from the segment that started at the 6-conflict state and reached zero at iteration 159 966 after 2 770 seconds, its conflict count falling as $6 \to 5 \to 4 \to 3 \to 2 \to 1 \to 0$. The colours moved a long way in the process: the final colouring differs from the 6-conflict state it started from in 1 032 edges, and from a 1-conflict colouring found in a parallel lane in 1 226 edges. Two colourings that are each within one monochromatic clique of being valid can thus be separated by more than a thousand edge changes, so the successful colouring is not a small repair of anything the search had already seen. For comparison, the same engine had earlier reduced an 11-conflict state to 2 and then made no further progress for two thousand seconds.

What mattered. Two things, in our assessment.

First, *restarts from the best state with fresh seeds.* A single long segment is misleading here. In one run the improvement rate collapsed to 0.003 conflicts per 100 seconds at 97 conflicts, a profile that reads as convergence to a floor. It was not: restarting from that same colouring with new seeds reached 41 conflicts within two hours, and three further rounds reached zero. Every round of fresh seeds found a deeper minimum than the last. Any assessment of this kind of search should be made per round, not per segment.

Second, the *last conflicts are not the cheapest.* One lane of the final round reached a 2-conflict state consisting of a single red K_8 and a single blue K_{10}; an exhaustive check of all 73 edges of those two cliques, together with the intersection condition for a second flip, showed that no single flip and no pair of flips removes both, so a repair had to change at least three coordinated edges. Breaking a red conflict turns those edges blue and so creates blue conflicts, and conversely, which is why the two ends pull against each other. The descent from 29 814 to 6 was therefore not the difficult part: it was the last few conflicts that required the search to leave the neighbourhood it had been exploring, and the successful segment spent 2 770 seconds and 159 966 iterations on its final six.

Conversely, we found no use for a donor-crossover operator at this scale. Two segments running a section-crossover variant, given the same budget from the same states, failed to improve on the plain patch move at 96 conflicts and at 6 conflicts, while the plain lanes improved at both. This is consistent with [1], where crossover is credited with closing a state at 6–7 conflicts after the repair stage had plateaued, but we did not reproduce that advantage here.

5 On the use of AI

The system. The computational work was carried out by *Open Math Model*, running DeepSeek V4.1 Flash — a model supplied by DeepSeek — at maximum reasoning effort, operating an ordinary Linux virtual machine with four cores. The agent wrote all programs, chose the search strategy, ran the experiments and did the bookkeeping; the author set the goal, fixed the compute budget and decided which cells to pursue. This is the same division of labour reported in [1].

The screening swarm. The first stage ran as an *Ultramath* swarm: eight identical agents (labelled `m1` to `m8`) attacking one shared problem concurrently. Each was required to register a distinct line of attack before starting — a duplicate registration is refused — and the agents exchanged findings on a common board rather than working in isolation. The problem given to the swarm was to apply the add-a-vertex-then-repair method of [1] to as many cells of Table 2.1 as possible and to determine, by measurement rather than assertion, which cells it can move. The run lasted four hours with a budget of one hundred model steps per member, on the same four cores used later for the present computation.

system	Open Math Model
model	DeepSeek V4.1 Flash
supplier	DeepSeek
reasoning effort	maximum
swarm size	8 agents, one shared problem, 10 distinct directions opened
duration	4 hours on 4 cores
outcome	8 submissions, all accepted after independent checking

What the swarm contributed. Four things, all of which are used in this paper.

1. An *exact twin-insertion table* for nine cells — for every vertex v and both colours of the joining edge, the number of conflicts created by copying v — covering the record colourings

then available for those cells, several of which are due to Exoo and Tatarevic [4]. This is the quantity that determines the starting point of §4, and its minimum over the 688 choices for the $(8, 10)$ cell is 29 814.

2. *Reusable engines.* The exact-conflict repair program used for the descent in §4 was written during the swarm, as was a second engine implementing the preparation, risk-chain and crossover stages of [1], a depth-bounded exhaustive repair checker, and a twin-scanning utility.
3. A *bounded-distance screen*: a CEGAR-SAT encoding that decides, for increasing k, whether any valid colouring of the target order lies within Hamming distance k of a given +1 state. Applied to six cells, it proved that no valid colouring lies within 9 to 17 edge changes of the corresponding state — a statement no heuristic can make — and thereby established that those cells cannot be moved from those starting colourings by this method at all, however long it runs. The screen is reproduced in Appendix A.
4. A *negative result about search heuristics*: the conflict count of the +1 state does not rank how far a cell is from a solution. One cell with 72 conflicts after the +1 step needed only 9 edge changes, while another with 6 needed 13. Every agent in the swarm, and the author's own initial reading, ranked the cells by conflict count; that ranking was wrong, and the $(8, 10)$ cell — which looked among the least promising, with 29 814 initial conflicts — is the one that moved.

The final campaign. The swarm finished with no cell moved. The descent reported in §4 was run afterwards by the agent alone, using the engine from item 2 above, in six rounds of four parallel single-threaded lanes over approximately eight hours of wall-clock time. The AI's role in the two stages differed in kind: the swarm was useful for breadth and for producing tools and negative results, while the positive result came from a long, narrow, single-purpose campaign that no member had the budget to run and that the swarm's own screening had not identified as the most likely to succeed.

Verification by the agent, not by the model. Every numeric claim in this paper was recomputed after the fact, outside the search process and largely outside the code that produced it, before being recorded: the witness by four independent checkers, the twin tables by three separate implementations, the distance bounds against positive controls at a known distance, and the search trajectories against exact conflict counts. Two errors in the intermediate records were caught this way. A depth-bounded exhaustive repair checker reported a repair from a 2-conflict state at $n = 345$ and wrote back an unchanged matrix; an independent enumeration of all 73 candidate first flips showed that no 1- or 2-flip repair existed there, and that checker's large-n output is not used anywhere in this paper (it is reliable against controls at $n = 40$ and $n = 58$). Separately, an earlier reading of the search described a descent that had stalled at 97 conflicts as approaching a floor; it was seed exhaustion, and the descent resumed as soon as the seeds were renewed, as described in §4.

6 What the computation does not show

- The upper bound is unchanged; $R(8, 10) \leq 4402$ [2] is far from 346. Nothing here bears on the value of $R(8, 10)$, only on its lower end.
- No statement is made about the existence of $(8, 10)$-colourings of K_{346}. A failed search for one would not be evidence of its non-existence, and we did not run that search.

- The colouring is not presented as canonical or as one of a known family. It was found by stochastic search; whether it is isolated among 345-vertex colourings is open.
- The bounded-distance screen of Appendix A produced no bound at any of the cells it covers, and it did not predict the present result: no such screen finishes at $n = 345$, so the $(8, 10)$ cell was outside what it could decide in either direction.

7 Reproduction

Artefacts. The colouring is deposited in two encodings:

matrix, 345 rows of `0/1` (1 = red)	`r810-n345-k8-free-k10-free.txt`
graph6 of the red graph	`r810-n345-k8-free-k10-red.g6`
SHA-256 of the matrix	`0af4590108642651e3999994b379235 f8b126bb9eb1fdfd36fb450ef47935b12`
SHA-256 of the graph6 file	`4135f58f2cb289bfa0ab02ca024e1a2 d7e5f695aa505e96fa88b4bde54158d62`

Splitting each hash across two lines is a typesetting device only; the two halves are one string.

Search. The search is single-threaded C. The run that produced the colouring was invoked as

```
engine start.txt 8 10 0 run 20000000 32 222 3600 1000 out 1.607 64
```

from a 345-vertex input with 6 conflicts, where the arguments are, in order, the input matrix, the forbidden red and blue clique sizes, a colour-convention flag, the mode, an iteration cap, the candidate sample size, the random seed, a CPU-second budget, the stall threshold, the output prefix, the red weight and a cap on the number of conflicts a flip may create. Inputs are matrices with a zero diagonal and 1 denoting a red edge; the colour convention must be stated explicitly, since published witnesses differ in it, and a matrix inverted by hand to change convention acquires a non-zero diagonal and will be mis-counted by any clique enumerator that assumes otherwise.

A How far a repair must travel: an exact bounded-distance screen

The method studied in this paper is a repair: from a colouring that is not yet valid, change edges until no forbidden clique remains. Whether that is possible at all is a question about distance, and inside a Hamming ball the question is decidable exactly. This appendix records the answers for six smaller cells, obtained during the screening stage described in §5. They are the reason the present cell was the one worth pushing, and they delimit the method in a way that no search can.

The question

Let B be a colouring of K_{n_0+1} obtained by adding one vertex to a valid (r, s)-colouring of K_{n_0} — a twin, in the sense of §4. For $k = 0, 1, 2, \ldots$ ask:

is there an (r, s)-colouring of K_{n_0+1} at Hamming distance at most k from B?

An unsatisfiable instance at bound k is a proof that *no* sequence of at most k edge changes repairs B, whatever heuristic is applied. It is therefore a statement about the cell, not about a program.

The encoding

One Boolean variable per edge, with 1 meaning red. The constraints are that no red K_r and no blue K_s exist. The red side is usually small enough to write out in full; the blue side is not — for $n = 40$ there are $\binom{40}{10} = 8.5 \times 10^8$ ten-sets — so the clique constraints are added lazily: solve, enumerate every violated clique exactly, add the clause forbidding that clique, and re-solve. The distance bound is imposed by a totalizer over flip variables $d_e \leftrightarrow (x_e \neq B_e)$, asserting $\sum_e d_e \leq k$.

Two solvers were used, CaDiCaL 1.5.3 and MiniSat 2.2, and they agree on every bound that both completed.

A positive control is mandatory

An encoding that is too strong returns "unsatisfiable" for every input, which looks identical to a correct proof. The screen was therefore validated against a case whose answer is known: take a valid $(4, 6)$-colouring of K_{35}, flip 5 of its edges at random, and require the encoder to return satisfiable at $k = 5$ with a witness that re-verifies to zero conflicts, and unsatisfiable at $k = 4$. Both behaviours were observed, and the returned witness was checked by an independent enumerator. Without such a control a distance bound is worthless.

Results

The column k^* is the largest k for which unsatisfiability was proved, so a repair from that state needs at least $k^* + 1$ edge changes.

cell	best twin	conflicts after +1	k^*	a repair needs
$(5, 6)$	v_{56}, red	136	16	≥ 17 edge changes
$(4, 6)$	v_{31}, red	24	16	≥ 17
$(3, 10)$	v_{35}, red	6	12	≥ 13
$(4, 7)$	v_{15}, red	40	12	≥ 13
$(3, 11)$	v_{20}, red	8	10	≥ 11
$(4, 9)$	v_0, red	72	8	≥ 9
$(6, 8)$, $(6, 9)$, $(8, 10)$	—	338, 2 830, 29 814	none	undecided

For $(3, 10)$ a second, better starting state was also screened: a 40-vertex colouring with four red triangles and no blue K_{10}, obtained by choosing the new vertex's red neighbourhood rather than copying a vertex. No valid colouring lies within 10 edge changes of it.

Two conclusions

The method cannot move those six cells from those bases. For $(4, 6)$ this is decisive: every engine in the study, including one running the complete pipeline of [1] with preparation, risk chains and mutation, stalls at 12 conflicts, and the nearest valid 36-vertex colouring is at least 17 edge changes away. No amount of tuning, restarting or extra seeds closes a gap of 17.

Conflict count does not rank repairability. The $(4, 9)$ start has 72 conflicts and needs only 9 edge changes; the $(3, 10)$ start has 6 — the smallest in the table, and the natural first choice — and needs 13. Ranking these cells by how many conflicts the vertex addition creates, which is the obvious heuristic and the one adopted at the outset of this work, orders them wrongly. The $(8, 10)$ cell, which looked the least promising of all with 29 814 initial conflicts, is the one that moved.

Limits

The bounds are for the specific best-twin extension of each base colouring; they are not statements about the cells, and a colouring of the target order might exist far from those particular states. For $(6, 8)$, $(6, 9)$ and $(8, 10)$ no bound was completed — no rung finishes at those sizes — so for those cells nothing is claimed in either direction, including for the result of this paper.

References

[1] F. Cremer. Lower bounds for Ramsey numbers: $R(6, 8) \geq 135$ and $R(8, 10) \geq 345$. Preprint, arXiv:2610.12122 [math.CO], 8 October 2026. Colourings at `https://github.com/fritzcremer/ramsey-lower-bounds`, release v1.1.0.

[2] S. P. Radziszowski. Small Ramsey numbers. *Electronic Journal of Combinatorics*, Dynamic Survey DS1, revision 18 (24 April 2026), 149 pp.

[3] E. Kuznetsov. Computational lower limits on small Ramsey numbers. Preprint, arXiv:1505.07186v5 [cs.DM], 2016.

[4] G. Exoo and M. Tatarevic. New lower bounds for 28 classical Ramsey numbers. *Electronic Journal of Combinatorics*, 22(3):Paper P3.11, 12 pp., 2015. arXiv:1504.02403.

[5] A. Nagda, P. Raghavan, and A. Thakurta. Reinforced generation of combinatorial structures: Ramsey numbers. Preprint, arXiv:2603.09172v5 [math.CO], 2026.

[6] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.